

Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет программной инженерии и компьютерной техники

Направление подготовки: 09.03.04 – Системное и прикладное
программное обеспечение
Дисциплина «Вычислительная математика»

Лабораторная работа №4

Вариант 17

Выполнил

Храбров Артём Алексеевич

Проверил

Наумова Надежда Александровна

Санкт-Петербург, 2026

Цель

Найти функцию, являющуюся наилучшим приближением заданной табличной функции по методу наименьших квадратов.

1. Вычислительная реализация задачи

1.1 Исходные данные

Задана функция:

$$f(x) = \frac{2x}{x^4 + 17}, \quad x \in [0, 2], \quad h = 0.2$$

Выполним табуляцию функции на интервале $[0, 2]$ с шагом $h = 0.2$.

i	x_i	y_i
1	0.0	0.00000
2	0.2	0.02353
3	0.4	0.04617
4	0.6	0.06573
5	0.8	0.07958
6	1.0	0.08333
7	1.2	0.07480
8	1.4	0.05652
9	1.6	0.03327
10	1.8	0.00761
11	2.0	-0.01961

1.2 Построение линейного приближения

Уравнение функции ищем в виде:

$$P_1(x) = ax + b$$

:

$$\begin{cases} a \sum x_i^2 + b \sum x_i = \sum x_i y_i \\ a \sum x_i + b \cdot n = \sum y_i \end{cases}$$

- $\sum x_i = 11.0$
- $\sum x_i^2 = 15.4$
- $\sum y_i = 0.46033$
- $\sum x_i y_i = 1.19762$

$$\begin{cases} 15.4a + 11.0b = 1.19762 \\ 11.0a + 11.0b = 0.46033 \end{cases}$$

$$4.4a = 0.73729 \implies a \approx 0.1676$$

$$b \approx -0.1258$$

Линейное приближение:

$$P_1(x) = 0.1676x - 0.1258$$

1.3 Построение квадратичного приближения

Уравнение функции ищем в виде:

$$P_2(x) = ax^2 + bx + c$$

$$\begin{cases} a \sum x_i^4 + b \sum x_i^3 + c \sum x_i^2 = \sum x_i^2 y_i \\ a \sum x_i^3 + b \sum x_i^2 + c \sum x_i = \sum x_i y_i \\ a \sum x_i^2 + b \sum x_i + c \cdot n = \sum y_i \end{cases}$$

- $\sum x_i^3 = 39.6$
- $\sum x_i^4 = 110.0$
- $\sum x_i^2 y_i = 0.94919$

$$\begin{cases} 110.0a + 39.6b + 15.4c = 0.94919 \\ 39.6a + 15.4b + 11.0c = 1.19762 \\ 15.4a + 11.0b + 11.0c = 0.46033 \end{cases}$$

$$a \approx -0.0983, \quad b \approx 0.3392, \quad c \approx -0.0136$$

Квадратичное приближение:

$$P_2(x) = -0.0983x^2 + 0.3392x - 0.0136$$

1.4 Нахождение среднеквадратических отклонений

СКО рассчитывается по формуле:

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - P(x_i))^2}$$

Для линейной аппроксимации:

Вычислив сумму квадратов разностей $\sum (y_i - P_1(x_i))^2$, получаем $S_1 = 0.00835$:

$$\sigma_1 = \sqrt{\frac{0.00835}{11}} = \sqrt{0.000759} \approx 0.0276$$

Для квадратичной аппроксимации:

Сумма квадратов разностей $\sum (y_i - P_2(x_i))^2$ составляет $S_2 = 0.00173$:

$$\sigma_2 = \sqrt{\frac{0.00173}{11}} = \sqrt{0.000157} \approx 0.0125$$

1.5 Сравнение результатов

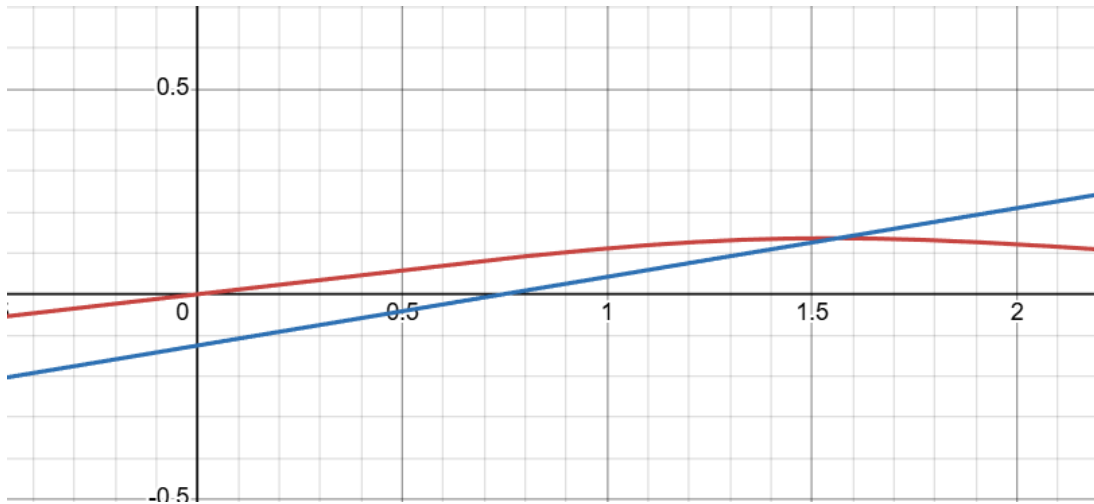
$$\sigma_2(0.0125) < \sigma_1(0.0276)$$

Вывод: Квадратичное приближение $P_2(x) = -0.0983x^2 + 0.3392x - 0.0136$ описывает заданный набор точек точнее, поэтому оно является наилучшим.

2. Графики

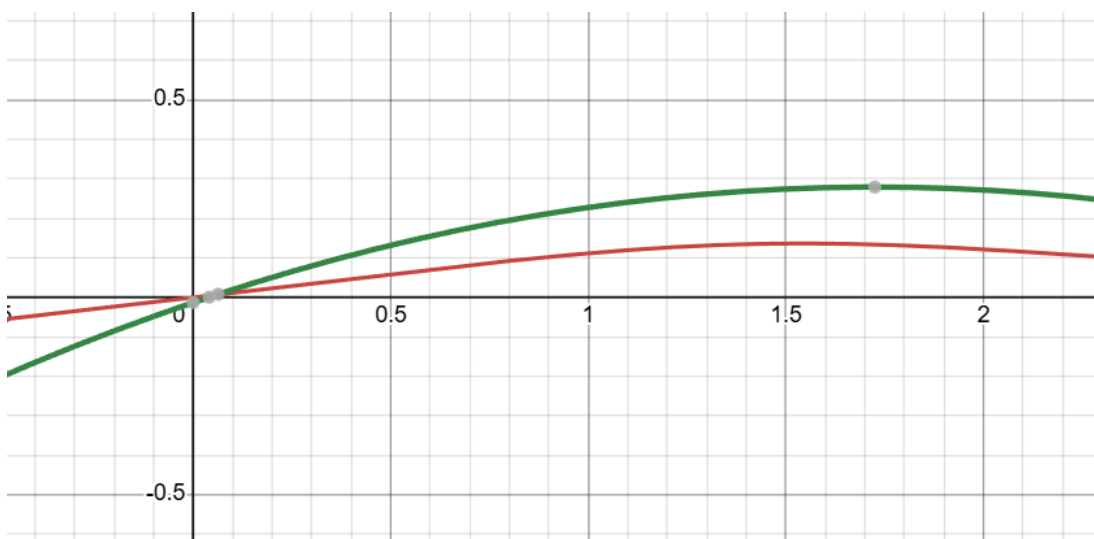
2.1 Линейное приближение

$$f(x) = \frac{2x}{x^4 + 17}, \quad g(x) = 0.1676x - 0.1258$$

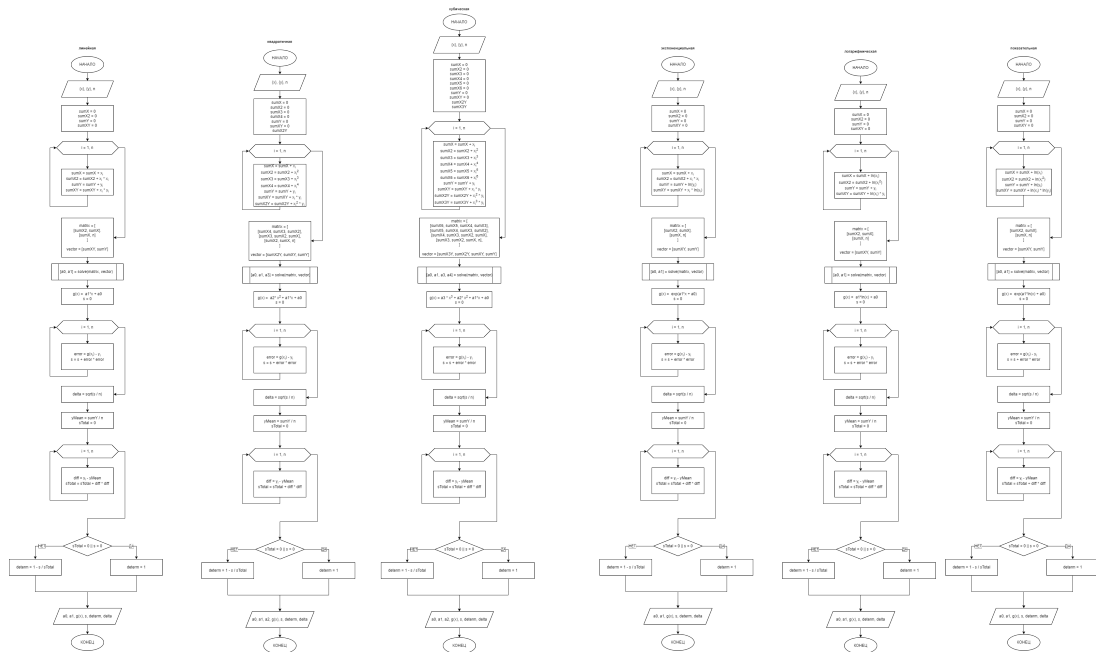


2.2 Квадратичное приближение

$$f(x) = \frac{2x}{x^4 + 17}, \quad g(x) = -0.0983x^2 + 0.3392x - 0.0136$$



3. Блок-схема



4. Листинг программы

[ССЫЛКА](#)

```

1 package arhr.tech.comp_math_lab2.services.solvers
2
3 import arhr.tech.comp_math_lab2.dto.ApproximationRequestDto
4 import arhr.tech.comp_math_lab2.dto.ApproximationResult
5 import arhr.tech.comp_math_lab2.dto.PointDto
6 import arhr.tech.comp_math_lab2.utils.SystemSolver
7 import java.math.BigDecimal
8 import java.math.MathContext
9 import java.math.RoundingMode
10 import kotlin.math.sqrt
11
12
13 abstract class AbstractApproximationSolver(
14     protected val type: ApproximationType
15 ) : ApproximationSolver {
16
17     protected val mc = MathContext.DECIMAL64
18     protected val systemSolver = SystemSolver()
19
20     protected abstract fun buildMatrix(points: List<PointDto>): Array<
21     Array<BigDecimal>>
22
23     protected abstract fun buildVector(points: List<PointDto>): Array<
24     BigDecimal>

```

```

23
24     protected abstract fun evaluate(coefficients: List<BigDecimal>, x:
BigDecimal): BigDecimal
25
26     protected abstract fun buildFormula(coefficients: List<BigDecimal>):
String
27
28     override fun solve(request: ApproximationRequestDto):
ApproximationResult {
29         val points = request.points
30         val n = points.size.toBigDecimal()
31
32         //
33         val matrix = buildMatrix(points)
34         val vector = buildVector(points)
35         val solution = systemSolver.solve(matrix, vector)
36         val coefficients = solution.toList()
37
38         //
39         val func = buildFormula(coefficients)
40
41         // S
42         var s = BigDecimal.ZERO
43         for (point in points) {
44             val yPred = evaluate(coefficients, point.x)
45             val error = yPred - point.y
46             s += error * error
47         }
48
49         //
50         val delta = if (s != BigDecimal.ZERO) {
51             sqrt(s.divide(n, mc).toDouble()).toBigDecimal()
52         } else {
53             BigDecimal.ZERO
54         }
55
56         // determ
57         val sumY = points.sumOf { it.y }
58         val yMean = sumY.divide(n, mc)
59
60         var sTotal = BigDecimal.ZERO
61         for (point in points) {
62             val diff = point.y - yMean
63             sTotal += diff * diff
64         }
65
66         val determ = when {
67             sTotal == BigDecimal.ZERO -> BigDecimal.ONE

```

```

68         s == BigDecimal.ZERO -> BigDecimal.ONE
69         else -> BigDecimal.ONE - s.divide(sTotal, mc)
70     }
71
72     // pearson
73     val pearson = if (type == ApproximationType.LINEAR) {
74         val sumX = points.sumOf { it.x }
75         val xMean = sumX.divide(n, mc)
76
77         var sumXdevYdev = BigDecimal.ZERO
78         var sumXdev2 = BigDecimal.ZERO
79         var sumYdev2 = BigDecimal.ZERO
80
81         for (point in points) {
82             val xDev = point.x - xMean
83             val yDev = point.y - yMean
84             sumXdevYdev += xDev * yDev
85             sumXdev2 += xDev * xDev
86             sumYdev2 += yDev * yDev
87         }
88
89         if (sumXdev2 != BigDecimal.ZERO && sumYdev2 != BigDecimal.
90 ZERO) {
91             sumXdevYdev.divide(
92                 sqrt((sumXdev2 * sumYdev2).toDouble()).toBigDecimal
93             ),
94             mc
95         )
96     } else {
97         BigDecimal.ZERO
98     }
99 } else {
100     null
101 }
102
103 return ApproximationResult(
104     type = type,
105     func = func,
106     s = s.round(mc),
107     determ = determ.round(mc),
108     delta = delta.round(mc),
109     pearson = pearson?.round(mc),
110     coffs = coefficients.map { it.round(mc) }
111 )
112
113 }
114
115 override fun supports(type: ApproximationType): Boolean = this.type
116 == type

```

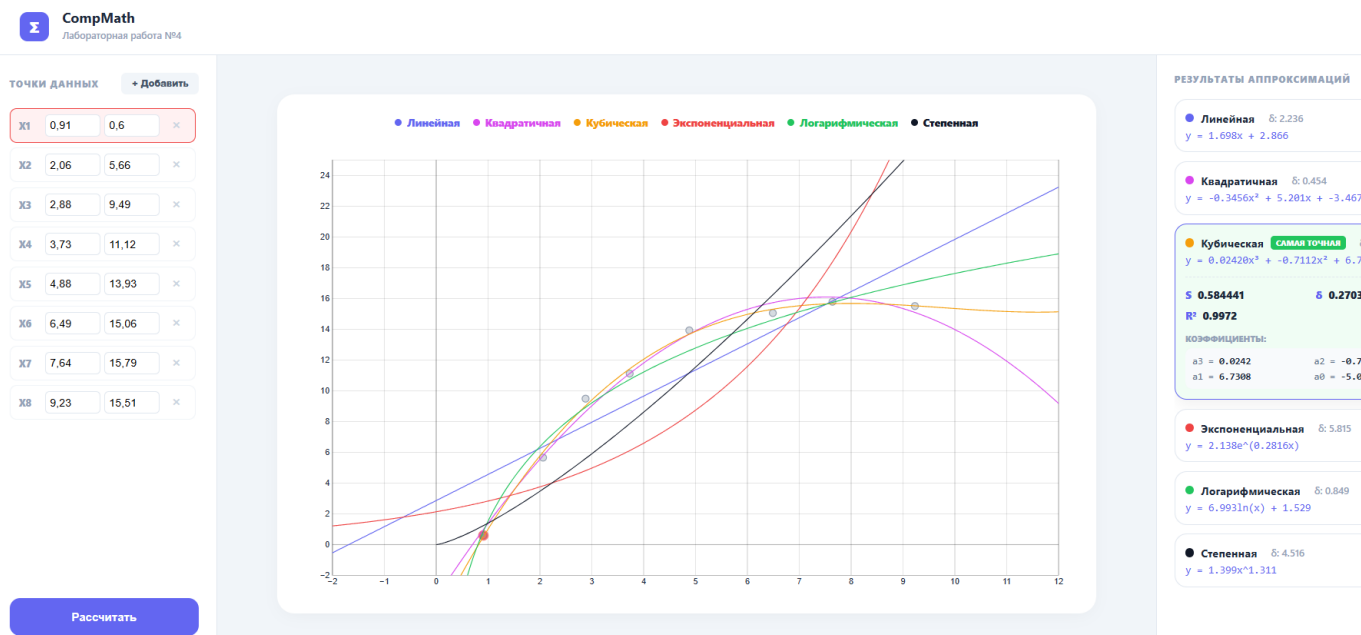


```

113
114
115     protected fun BigDecimal.round(): BigDecimal {
116         if (this.abs() <= BigDecimal("0.0000001")) return BigDecimal.
ZERO
117         return this.round(MathContext(4, RoundingMode.HALF_UP))
118     }
119
120     protected fun List<BigDecimal>.sumOf(selector: (BigDecimal) ->
BigDecimal): BigDecimal {
121         var result = BigDecimal.ZERO
122         for (element in this) {
123             result += selector(element)
124         }
125         return result
126     }
127
128     protected fun Int.pow(exp: Int): Int {
129         var result = 1
130         repeat(exp) { result *= this }
131         return result
132     }
133 }

```

5. Пример работы программы



Вывод

В ходе выполнения данной работы я изучил метод наименьших квадратов и реализовал его на языке Kotlin. Нашёл аппроксимации функции $f(x) = \frac{2x}{x^4+17}$. Квадратичное приближение оказалось точнее линейного.